Culture DevOps

Développer et exploiter, une seule chaîne

Séquence
SQ01

Partie
P01

Niveau
Mastère ESI

Objectifs

- Expliquer pourquoi développement et exploitation se sont opposés
- Décrire le cycle DevOps et situer une pratique ou un outil sur ce cycle
- Analyser un incident de livraison et proposer les pratiques qui l'évitent

Prérequis : aucun prérequis disciplinaire au-delà de la présentation du module.

ATTENTION

Vendredi, 17 h 30

Le développeur dépose la nouvelle version de « Voyages scolaires » par FTP, puis part en week-end.

Lundi 9 h, les parents appellent le lycée : la liste des voyages affiche une **erreur 500**.

Qui est responsable ? Et qu'est-ce qui a vraiment échoué ?

Deux métiers, deux objectifs

	Développement (Dev)	Exploitation (Ops)
Mission	Livrer des fonctions nouvelles	Garder le service disponible
Mesuré sur	Ce qui est livré	Ce qui tombe en panne
Le changement est	Le but	Le premier risque
Réflexe en cas d'incident	« Chez moi, ça marche »	« On gèle les mises en production »

Le mur de la confusion

- Le code est « jeté par-dessus le mur » à l'exploitation
- Les livraisons s'espacent, donc chacune devient **plus grosse et plus risquée**
- Un incident devient une recherche de coupable
- Personne ne voit la chaîne en entier

Chaque camp optimise sa part ; c'est le service rendu à l'utilisateur qui en paie le prix.

Le déploiement artisanal

Geste	Risque
Copie de fichiers par FTP	Version en ligne inconnue, aucun retour arrière
Migrations lancées « à la main »	Oubli, ordre faux, base incohérente
Configuration éditée sur le serveur	Serveur impossible à reconstruire à l'identique
Test : « j'ai regardé la page »	Les régressions passent en production
Personne ne surveille	Ce sont les utilisateurs qui découvrent la panne

DevOps

Une **culture** et un ensemble de **pratiques** qui rapprochent ceux qui développent et ceux qui exploitent une application, autour d'une chaîne de livraison **automatisée**, **mesurée** et dont ils partagent la **responsabilité**, du code jusqu'à la production.

DevOps n'est ni un outil, ni un poste, ni une équipe de plus.

D'où vient le mot

- **2008-2009** : Patrick Debois, consultant belge, cherche à réconcilier développeurs et administrateurs
- **Juin 2009** : à la conférence Velocity, Allspaw et Hammond présentent *10+ Deploys per Day: Dev and Ops Cooperation at Flickr*
- **Octobre 2009** : premiers *DevOpsDays* à Gand ; le mot-dièse `#devops` s'impose
- **2018** : *Accelerate* (Forsgren, Humble, Kim) relie ces pratiques à la performance des organisations

Qui construit exploite

> *« You build it, you run it. »*

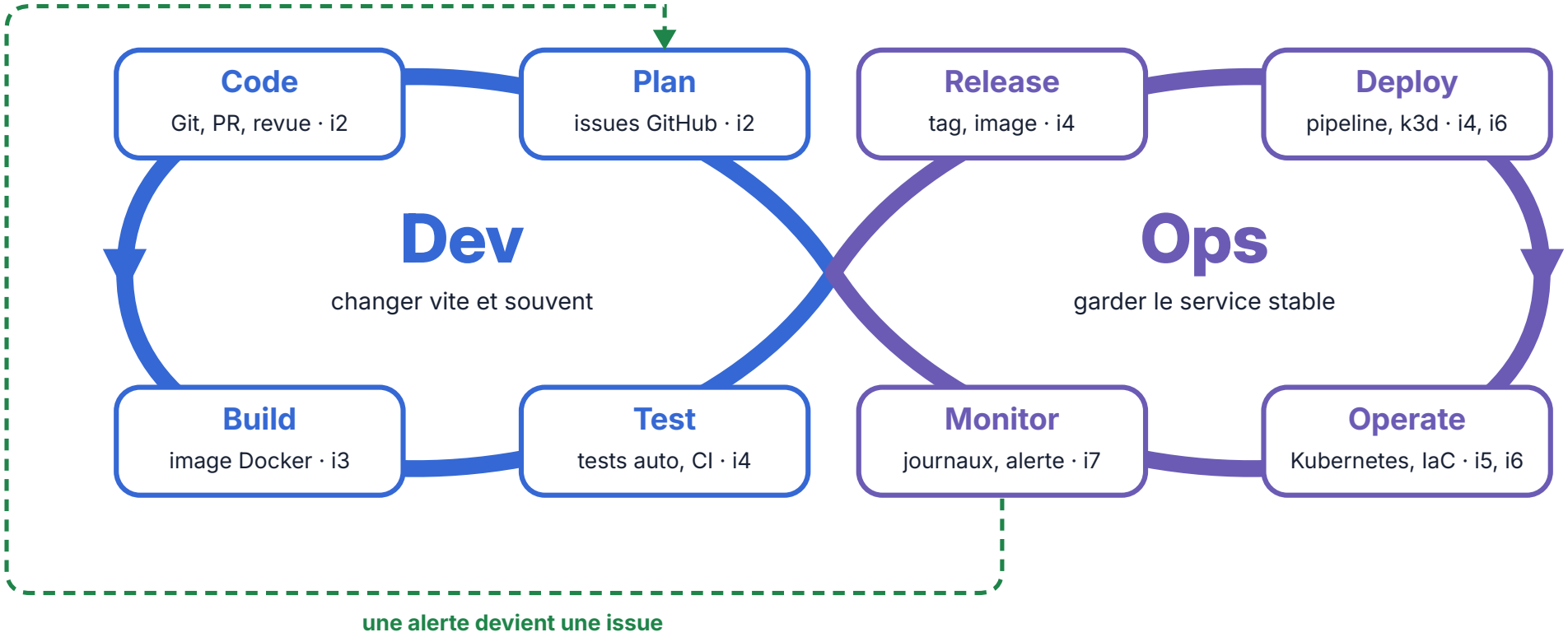
Werner Vogels, directeur technique d'Amazon, 2006 : l'équipe qui développe un service en assure aussi le fonctionnement en production.

CALMS : cinq piliers

Pilier	Ce qu'il change	Dans ce module
Culture	Responsabilité partagée, incidents sans recherche de coupable	Revue croisée, bilan d'incident
Automatisation	Plus de geste manuel répétitif dans la chaîne	GitHub Actions, Terraform, k3d
Lean	Petits lots, livrés souvent	Branches courtes, PR fréquentes
Mesure	Décider sur des faits	Journaux, disponibilité, alerte
Share (partage)	Code, outils et connaissances communs	Dépôt commun, README, PR

Le cycle DevOps, appliqué au fil rouge

Sécurité à chaque étape · secrets hors du dépôt, droits minimaux du pipeline, image vérifiée · i4, i7



Une seule chaîne, automatisée, dont l'équipe entière est responsable.

Fil rouge « Voyages scolaires » · iN = intervention du module où l'étape est travaillée

Intégration, livraison, déploiement continus

	Intégration continue (CI)	**Livraison continue**	**Déploiement continu (CD)**
Chaque modification	est fusionnée souvent, construite et testée	produit une version prête à livrer	part en production
Geste humain restant	la revue	**la décision** de livrer	aucun
Condition	tests automatiques fiables	CI + déploiement automatisé	livraison continue + surveillance

Mesurer la chaîne : les indicateurs DORA

Indicateur	Question	Mesure la
Fréquence de déploiement	Combien de livraisons en production ?	rapidité
Délai de mise en production	Du commit à la production, combien de temps ?	rapidité
Taux d'échec des changements	Quelle part des livraisons provoque un incident ?	stabilité
Temps de rétablissement	Combien de temps pour revenir à un service normal ?	stabilité

Les équipes les plus performantes vont **à la fois** plus vite et plus sûrement.

Qui fait quoi ?

Étape	Dev	Ops	Ensemble
Code, revue	écrit, relit	relit ce qui touche l'exploitation	règles de la PR
Tests	les écrit		les rend bloquants dans la CI
Image, pipeline	Dockerfile	droits, secrets, cibles	le pipeline lui-même
Production	corrige	surveille, alerte	astreinte, bilan d'incident

ATTENTION

Ce que DevOps n’est pas

- Une **équipe « DevOps »** de plus entre les deux autres : le mur reste
- Les **développeurs seuls** en production, sans compétence d’exploitation
- Un **outil** à acheter : Jenkins ou Kubernetes sans la culture ne changent rien
- La **suppression des contrôles** : on les automatise, on ne les retire pas

ACTIVITÉ

Exercice 1 — Placer sur le cycle

5 min · seul · sur papier

À produire : l'étape du cycle de chacune des 8 actions du support A4, partie 1.

1. Associer chaque action à une étape
2. Pour chacune, dire si elle relève de Dev, d'Ops ou des deux

ACTIVITÉ

Exercice 2 — Analyser l'incident du vendredi

10 min · seul, puis confrontation à trois · sur papier

À produire : un tableau *défaillance · pratique qui l'évite · étape · indicateur DORA dégradé*.

1. Relever au moins trois défaillances dans le récit du support
2. Pour chacune, proposer une pratique DevOps précise

ACTIVITÉ

Exercice 3 — La chaîne de Voyages scolaires

15 min · à trois · une seule production

À produire : la chaîne cible du projet, de la PR à l'alerte, en six étapes au plus.

1. Pour chaque étape : ce qui se passe **automatiquement** et l'outil
2. Qui est prévenu si l'étape échoue ?
3. Confronter au déroulé du module : qu'avez-vous oublié ?

ACTIVITÉ

Diagnostic

10 min · seul · sans aide · non noté

À produire : les 8 réponses du questionnaire, support A4 partie 1.

Git, Linux, Docker : il sert à régler les prochaines interventions sur vos acquis.

À RETENIR

À retenir

- DevOps : une **culture** de responsabilité partagée, outillée par l'**automatisation**
- Un cycle continu : Plan, Code, Build, Test, Release, Deploy, Operate, Monitor
- CI, livraison continue, déploiement continu : trois niveaux d'automatisation
- On mesure vitesse **et** stabilité (DORA), qui progressent ensemble

Première étape de la chaîne : intégrer le code proprement, avec Git et GitHub.